Training NeuralNector with Generative Adversarial Networks on Flowers 102

Issac Roy [1]

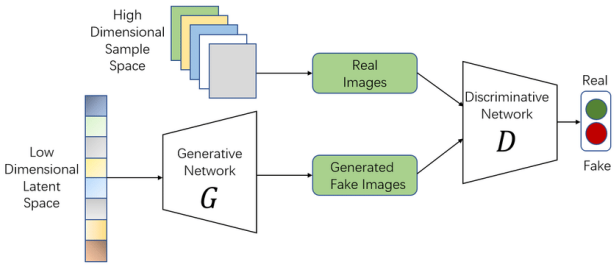


Figure 1. High-level GAN setup: the generator maps latent noise to images; the discriminator scores real vs. fake.

Abstract

This work trains DCGAN-style generative adversarial networks on MNIST and Oxford Flowers 102, progressing from a compact architecture for 28×28 digits to a deeper convolutional generator and discriminator for 64×64 RGB flower images. The resulting flower generator is deployed in NeuralNector, an interactive game in which human players act as the discriminator on grids of real versus synthetic images. Training is discussed in light of practical difficulties including discriminator dominance, vanishing gradients, and mode collapse, together with stabilizing modifications to the optimization procedure.

1. Introduction

Generative adversarial networks (GANs) (Goodfellow et al., 2014) are a framework for learning a model of a data distribution by pitting two neural networks against each other. A *generator* G maps samples z from a simple prior (e.g., Gaussian noise) to synthetic data $G(z)$ in the same space as real examples x. A *discriminator* D is a classifier trained to distinguish real data from generated samples. Training alternates between updating D to assign high probability to real x and low probability to $G(z)$, and updating G to produce samples that fool D. Figure 4 sketches the adversarial setup and layer structure used in this work.

The original minimax objective (Goodfellow et al., 2014) is defined as follows:

$$\min_G \max_D \; \mathbb{E}_{x\sim p_{\text{data}}}[\log D(x)] + \mathbb{E}_{z\sim p_z}[\log(1-D(G(z)))]\,, \tag{1}$$

where $D(x) \in (0,1)$ is interpreted as the probability that x is real. In practice, G is often trained with a *non-saturating* loss that maximizes $\mathbb{E}_z[\log D(G(z))]$ instead of minimizing $\mathbb{E}_z[\log(1-D(G(z)))]$, which can mitigate vanishing gradients when D rejects fakes with high confidence (Goodfellow et al., 2014). Our implementation follows the standard recipe of optimizing D and G with alternating stochastic gradient steps on losses equivalent to binary cross-entropy between D's output and labels for real vs. fake batches (with additional stabilizers described in Section 2). However, we found that the discriminator can become too accurate too quickly, which yields vanishing gradients for G and stalled learning (Arjovsky & Bottou, 2017; Salimans et al., 2016). Our solution is to skip the discriminator update if the discriminator's loss dips below a certain threshold, allowing G to catch up.

GAN training is notoriously fragile. As mentioned above, the discriminator can become too accurate too quickly, which yields vanishing gradients for G and stalled learning. *Mode collapse* occurs when G maps many latent vectors to a small set of outputs, limiting diversity (Arjovsky & Bottou, 2017; Salimans et al., 2016). Figure 2 shows an example of mode collapse on the Flowers 102 dataset where the generator only produces white images. Other pathologies include oscillations and sensitivity to hyperparameters (Arjovsky & Bottou, 2017). These issues motivated a large body of follow-on work (e.g., improved optimization and evaluation (Salimans et al., 2016; Arjovsky et al., 2017)); in this project we address practical instability with architectural scaling and training heuristics rather than attempting to formulate a new objective.

We began with the MNIST dataset and a compact convolutional architecture in `GAN_MNIST.ipynb`: a shallow

[1]University of California, San Diego, La Jolla, USA. Correspondence to: Issac Roy <iroy@ucsd.edu, ucsd>.

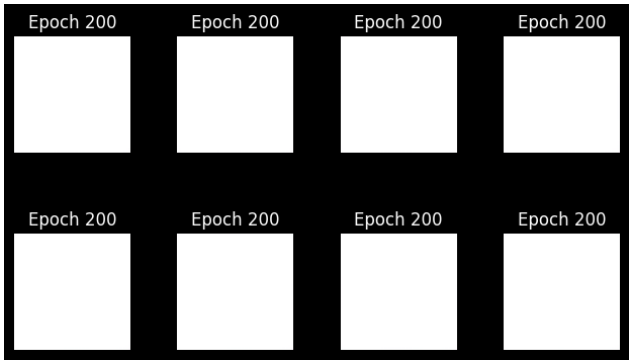


Figure 2. Mode collapse in the generator on the Flowers 102 dataset causes only white images to be generated.

discriminator with convolutions, pooling, and fully connected layers, and a generator built from a linear projection to a small spatial grid followed by transposed convolutions to produce 28×28 grayscale digits. We then moved to the Oxford Flowers 102 dataset (Nilsback & Zisserman, 2008), which is substantially harder: natural color images at higher resolution require a deeper DCGAN-style design with more layers and 64×64 RGB outputs, developed in `GAN_pytorch.ipynb`. On Flowers we observed vanishing gradients, mode collapse, and a discriminator that reached near-optimality too early, suppressing learning in G. We mitigated these effects with practices such as batch normalization in the deeper networks, label smoothing, asymmetric learning rates for G and D, and selective updates that skip or repeat discriminator steps when its loss indicates it is overpowering the generator; details appear in Section 2. The resulting generator powers NeuralNector, an interactive game in which human players act as discriminators on grids of real vs. GAN-generated flowers.



Figure 3. Samples from the generator at selected training epochs (flowers).

2. Methods

All models are implemented in PyTorch with the PyTorch Lightning training framework, using manual optimization so the generator and discriminator can be updated on different schedules with distinct learning rates.

2.1. Datasets and splits

MNIST. Grayscale digits are resized to 28×28. Inputs use the standard per-channel normalization to match the mean and variance. The training partition is split into approximately $55{,}000$ training and $5{,}000$ validation examples; the held-out test set is reserved for qualitative checks. This setting is used to validate the training loop and a lightweight convolutional architecture before scaling to natural images.

Oxford Flowers 102 (Nilsback & Zisserman, 2008). Images are resized and cropped to 64×64 RGB. Pixel values are mapped to $[-1, 1]$ via normalization with mean and standard deviation 0.5 per channel, which matches the `tanh` output range of the generator. The dataset provides only about $1{,}000$ images in each of the official training and validation splits; to increase per-epoch diversity and exposure to the class distribution, the training and validation splits are concatenated for optimization. The official test split isn't used during training and is kept for evaluation in Section 3. When training, optional data augmentation (random crops, flips, jitter, etc.) further increases variability for the flower domain.

2.2. Architectures: MNIST vs. Flowers

The same conceptual GAN objective (Equation (1)) is used throughout; the capacity and inductive bias of the networks are scaled with dataset difficulty. Figure 4 summarizes the layer stacks for both regimes, with G and D drawn side by side within each panel.

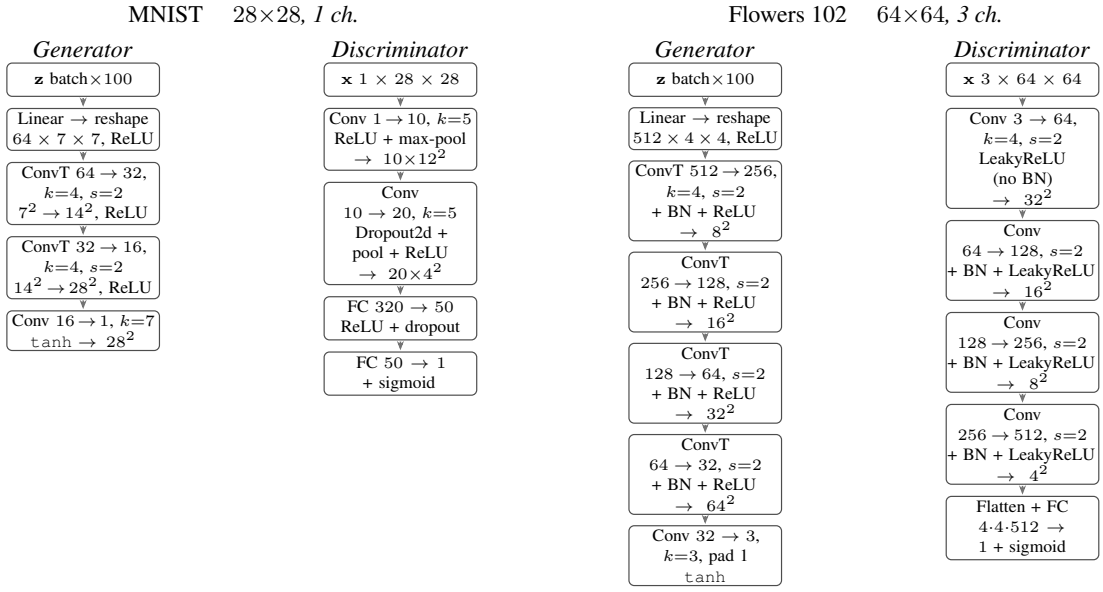


Figure 4. Neural architectures (`models.py`). *Left:* MNIST—G and D side by side; shallow D (pooling, dropout, no BN); G without BN. *Right:* Flowers—DCGAN-style D (BN except first conv, LeakyReLU) and G (transposed convs + BN + ReLU, `tanh`).

MNIST (28×28, 1 channel). The discriminator is a small CNN: two convolutional layers with 5×5 kernels, max pooling, dropout on the second conv block, and two fully connected layers ending in a single logit passed through a sigmoid. This shallow design is sufficient for aligned digits and trains quickly while limiting overfitting on a simple domain. The generator maps Gaussian noise of dimension $z \in \mathbb{R}^{100}$ through a linear layer to a $7 \times 7 \times 64$ tensor, applies two transposed convolutions to reach spatial size 28×28, and a final convolution with `tanh`. ReLU activations are used in the generator, and ReLU with pooling in the discriminator.

Flowers (64×64, 3 channels). Natural color flowers exhibit richer texture and spatial structure, so both networks follow a DCGAN-style deep convolutional layout with strided convolutions in D and transposed convolutions in G. The discriminator stacks four strided 4×4 convolutions with channel widths $64 \to 128 \to 256 \to 512$, batch normalization on all but the first layer (a common practice so the input statistics are not whitewashed), LeakyReLU (0.2) activations, and a linear layer to one output with sigmoid. BN and LeakyReLU help stabilize gradients when D must discriminate fine-grained natural detail; the first conv remains unnormalized so the network can learn low-level edge statistics directly from pixels. The generator projects $z \in \mathbb{R}^{100}$ to $4 \times 4 \times 512$, then upsamples through four transposed convolution stages ($512 \to 256 \to 128 \to 64 \to 32$ feature maps) each followed by BN and ReLU, and a final 3×3 convolution to three channels with `tanh`. BN in G reduces internal covariate shift across upsampling stages and tends to improve stability when many layers are stacked.

Convolutional, transposed-convolutional, and linear weights are initialized from $\mathcal{N}(0, 0.02^2)$ with zero bias initialization, following common GAN practice.

2.3. Loss and alternating updates

The discriminator is trained to minimize binary cross-entropy between its sigmoid output and soft targets for real vs. fake batches. Label smoothing with parameter 0.1 replaces hard labels 1 and 0 for real and fake with 0.9 and 0.1, which discourages D from becoming overconfident and mitigates vanishing gradients through the generator when fakes are trivially rejected (Salimans et al., 2016). The generator is trained to fool D, using BCE against targets of "real" (1) for generated samples (non-saturating form, consistent with Equation (1) and the discussion in the introduction).

2.4. Optimization and adaptive balancing

Both networks use Adam with betas $(0.5, 0.999)$. The base learning rate is 2×10^{-4} for the hyperparameter `lr`, but asymmetric effective rates are applied: the generator uses $\texttt{lr}_G = 2\times\texttt{lr}$ (4×10^{-4}) and the discriminator $\texttt{lr}_D = \frac{1}{2}\times\texttt{lr}$ (1×10^{-4}). The intent is to slow D and accelerate G so the discriminator does not reach optimality too quickly which is a regime in which G receives little useful gradient and training stalls.

Additional adaptive scheduling is implemented in the training step: if the averaged discriminator loss d_{loss} falls below 0.6, the discriminator update for that step is skipped (D is already winning too easily; updating it further would harm G). If d_{loss} falls below 0.4, the generator is updated twice

Table 1. Default hyperparameters for Flowers 102 training (unless varied in Section 3).

SETTING	VALUE
LATENT DIMENSION $\lvert\mathbf{z}\rvert$	100
BASE LR	2×10^{-4}
lr_G / lr_D	$2\times$ / $0.5\times$ BASE
LABEL SMOOTHING	0.1
ADAM (β_1, β_2)	(0.5, 0.999)
SKIP D STEP IF $d_{\text{loss}} <$	0.6
EXTRA G STEP IF $d_{\text{loss}} <$	0.4
FLOWERS BATCH SIZE	64
OUTPUT ACTIVATION	TANH ON G, SIGMOID ON D

in one step (second draw of z) so G can recover when D is very strong. Together with label smoothing and asymmetric learning rates, this implements a simple heuristic schedule that pauses or emphasizes updates based on the relative strength of D and G, without changing the underlying minimax game.

Typical batch sizes are 128 for early MNIST experiments in `GAN_MNIST.ipynb` and 64 for Flowers in `GAN_pytorch.ipynb`, subject to GPU memory.

2.5. Selecting fakes for NeuralNector

After training, synthetic flowers for the web game are produced by sampling the generator and ranking candidates by the discriminator's output, interpreted as the model's estimated probability that an image is real. A large pool of candidates is generated (and optionally merged with previously generated images), all are scored under a fixed checkpoint, and the top 500 by discriminator score are written to the game's fake image archive. This heuristic filters out egregious failures and mode-collapsed samples that D confidently rejects, improving the quality of the human-facing "fake" set at the cost of biasing the pool toward samples the current D finds plausible. The same trained `gan_flowers_final.ckpt` checkpoint is loaded in production for inference.

3. Experiments

We report two complementary evaluations: (i) a *metrics-driven* Flowers 102 study reporting Fréchet Inception Distance (FID) (Heusel et al., 2017), Kernel Inception Distance (KID), and Inception Score (IS) (Salimans et al., 2016) via the `torch-fidelity` library on held-out test reals, and (ii) a lightweight MNIST ablation focused on optimization behavior (BCE losses only). The Flowers protocol is implemented in `GAN_experiments.ipynb`: five DCGAN runs of 150 epochs at batch size 64 ($\approx 4.8 \times 10^3$ optimizer steps per run), targeting roughly one hour of GPU time on a typical Colab instance, which is still far shorter than the multi-thousand-epoch production training in `GAN_pytorch.ipynb`. Hyperparameters follow Table 1 unless an ablation overrides them. All Flowers metrics use $N = 2048$ generated samples matched against the first N images of the official *test* split (deterministic resize/center crop to 64×64, no augmentation), so training never sees the reference set. FID/KID/IS rely on Inception-v3 features and are most reliable as *relative* comparisons across rows in Table 2; at 64×64 and with a single seed they should not be read as absolute benchmarks.

3.1. Flowers: FID, KID, and Inception score

Table 2 reports five runs from a single Colab execution (NVIDIA A100-40GB; $\approx$ 57 minutes wall time summing training and metric computation). Configurations are `FL1_baseline` (default stabilizers and augmentation), `FL2_no_aug`, `FL3_always_train_D` (no adaptive skipping of D), `FL4_symmetric_lr` ($\mathrm{lr}_G = \mathrm{lr}_D$), and `FL5_no_label_smooth`. We log BCE-style L_G, L_D, and batch d_{acc} (fraction of reals above 0.5 and fakes below 0.5).

By FID and KID, `FL4_symmetric_lr` is strongest on this seed (FID 211, KID 0.180), with the highest IS as well—noting that IS is less directly aligned with natural flowers than with ImageNet-like class structure. `FL2_no_aug` is weakest (FID 308, IS 1.75), supporting the use of augmentation for this small-data regime. `FL5_no_label_smooth` improves over `FL1` on FID/KID despite similar late-epoch losses, while `FL3_always_train_D` lands between `FL1` and `FL2` on FID without the catastrophic MNIST loss split seen when D is always updated on digits (Table 3); Flowers optimization appears less brittle to that ablation on this budget. Overall, metrics disagree with a naive reading of BCE alone (e.g., `FL4` has higher L_G than `FL1` but better FID), underscoring the value of distribution metrics alongside losses.

3.2. MNIST training ablations

On MNIST we train for 12 epochs at batch size 128 with the 28×28 architecture from Section 2. Table 3 summarizes final losses and means over the last three epochs. Several ablations leave both losses in a similar band (roughly $L_G \approx 0.59$–0.70, $L_D \approx 0.58$–0.59): removing label smoothing (`M2`), using symmetric learning rates for G and D (`M3`), reducing the latent dimension to 50 (`M5`), and raising Adam's β_1 to 0.9 (`M6`). These differences are modest on this budget and should be interpreted cautiously because GAN losses need not correlate monotonically with sample quality.

The clearest signal is `M4_always_train_D`, which disables the adaptive rule that skips discriminator updates when $d_{\text{loss}} < 0.6$ (implemented as a skip threshold of -1). Here the discriminator loss falls to about 0.34 while the gener-

Table 2. Flowers 102 generative metrics (150 epochs, batch 64, $N = 2048$ fakes vs. test reals, one seed). Lower FID/KID is better. d_{acc}: final epoch; $\bar{d}^{(3)}_{\text{acc}}$: mean over last three epochs. Source: `GAN_experiments.ipynb` / `EXPERIMENTS_JSON`.

RUN	FID↓	KID	IS	L_G	L_D	d_{acc}	$\bar{d}^{(3)}_{\text{acc}}$
FL1_BASELINE	279.1	0.272	2.23	0.778	0.704	0.505	0.511
FL2_NO_AUG	307.8	0.331	1.75	1.079	0.643	0.662	0.672
FL3_ALWAYS_TRAIN_D	302.2	0.301	2.08	0.753	0.705	0.499	0.510
FL4_SYMMETRIC_LR	211.0	0.180	3.43	1.180	0.643	0.660	0.667
FL5_NO_LABEL_SMOOTH	239.9	0.214	2.34	0.770	0.706	0.489	0.492

Table 3. MNIST ablations (12 epochs, batch 128). L_G, L_D: losses at the final epoch; $\bar{L}^{(3)}$: mean over the last three epochs.

RUN	L_G	L_D	$\bar{L}^{(3)}_G$	$\bar{L}^{(3)}_D$
M1_BASELINE	0.697	0.581	0.689	0.584
M2_NO_LABEL_SMOOTH	0.612	0.578	0.608	0.580
M3_SYMMETRIC_LR	0.594	0.585	0.592	0.586
M4_ALWAYS_TRAIN_D	2.258	0.342	2.239	0.343
M5_LATENT_50	0.683	0.585	0.684	0.585
M6_BETA1_0.9	0.681	0.585	0.682	0.585

ator loss climbs to about 2.26, with a monotonic upward trend in L_G after the first epoch. This pattern is consistent with discriminator dominance: D becomes too confident too quickly, which weakens the learning signal for G under the non-saturating objective. The default schedule (`M1_baseline`), which can pause D, avoids this failure mode on the same epoch budget. Together, these results support the narrative in the introduction: the heuristic balancing of G and D updates is not merely cosmetic; when removed, optimization behavior changes sharply even on a simple dataset.

For a qualitative view of the flower generator used in deployment after long training, see Figure 5 (trained and evaluated as described in Section 4).

3.3. Limitations

The Flowers metrics use a *single* random seed, $N = 2048$ samples per side, and 64×64 inputs, so FID/KID/IS have non-trivial variance; we use them for *relative* ranking across ablations in Table 2. The MNIST block remains loss-only. Neither study replaces a full multi-seed sweep or human evaluation at scale.

Table 2 fixes the base learning rate at 2×10^{-4} (DCGAN-style default in Table 1) and does not include a full grid over learning rate or optimizer choice (e.g., Adam vs. SGD) at matched compute; `FL4_symmetric_lr` instead varies the *relative* rates lr_G and lr_D. Repeating the FID protocol across seeds and over $\{10^{-4}, 2 \times 10^{-4}, 4 \times 10^{-4}\}$ would better match the breadth of tuning but would be much more compute-intensive.

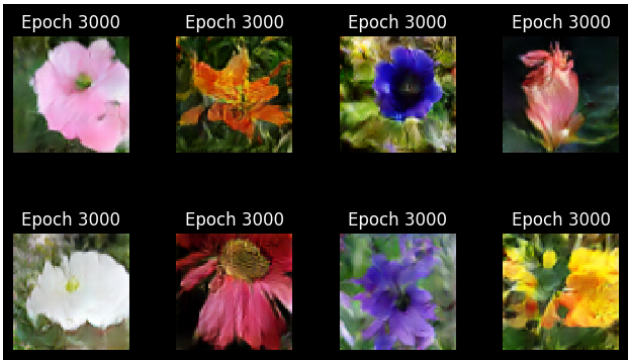


Figure 5. Qualitative samples from the production flower generator (baseline recipe, 3000 epochs).

4. Conclusion

We implemented DCGAN-style adversarial training for MNIST and Oxford Flowers 102, scaling architecture and data pipeline with dataset difficulty, and documented practical failure modes (discriminator dominance, mode collapse, vanishing gradients) familiar from the GAN literature. Table 2 shows that the baseline was around the median for many metrics, while symmetric learning rates achieved the best FID/KID over 150 epochs, and removing augmentation hurt all reported metrics.

The deployed game uses the *asymmetric* baseline in Table 1, trained 3000 epochs ($\approx$ 4 GPU-hours on an NVIDIA A100-40GB, `GAN_pytorch.ipynb`). With the same $N = 2048$ FID protocol as Table 2 (generated vs. held-out test reals), that checkpoint reaches FID 50.3, far below the 150-epoch runs (best: $\approx$ 211 for `FL4_symmetric_lr`), so most perceptual gain comes from long optimization. Figure 5 shows typical samples. We still prefer this recipe over the short-run winner because it was validated at long horizon and reflects the original goal of slowing D early. The generator powers NeuralNector, where human players discriminate real from synthetic images ranked in part by discriminator scores.

Future work could add multi-seed FID confidence intervals, sweep learning rates and thresholds more densely, and explore alternative objectives or regularizers while preserving the game's latency and data-pipeline constraints.

References

Arjovsky, M. and Bottou, L. Towards principled methods for training generative adversarial networks, 2017.

Arjovsky, M., Chintala, S., and Bottou, L. Wasserstein gan, 2017.

Goodfellow, I. J., Pouget-Abadie, J., Mirza, M., Xu, B., Warde-Farley, D., Ozair, S., Courville, A., and Bengio, Y. Generative adversarial networks, 2014.

Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., and Hochreiter, S. GANs trained by a two time-scale update rule converge to a local Nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.

Nilsback, M. and Zisserman, A. Automated flower classification over a large number of classes. In *Proceedings of the Indian Conference on Computer Vision, Graphics and Image Processing*, 2008.

Salimans, T., Goodfellow, I., Zaremba, W., Cheung, V., Radford, A., and Chen, X. Improved techniques for training gans, 2016.